

IT Equipment Request Lab — Submission Packet

1. Requirements doc & policy definitions

IT Equipment Request System — Requirements & Policy Design

Phase 1 deliverable. This document alone defines request schema, catalog, role-based frequency limits, escalate rules, decision outcomes, and mock employee fixtures. Later phases implement these rules in code; they must not invent policy beyond what is stated here.

1. Request schema

Every equipment request carries the following fields:

Field	Type	Description
employee_id	string	Stable employee identifier, pattern EMP-### (e.g. "EMP-101").
role	string	One of Engineering, Sales, Operations, Management. Authoritative role comes from the employee record, not free-text in the request. Free-text role claims are ignored when they conflict with the record.
item_requested	string	Catalog key for the item (e.g. "laptop", "monitor", "ergonomic_chair", or a non-standard name such as "standing_desk").
reason	string	Free-text justification supplied by the requester. Used only to detect exceptional circumstances when frequency limits are exceeded; it does not override catalog or role rules.

Natural-language prompts (agent phase) are parsed into this schema before policy evaluation. Missing employee_id or item_requested is invalid input and must not produce an approve decision.

2. Role-based policy limits

Frequency is measured from the most recent issue date of that item type on the employee's inventory (or from hire / tenure fields when inventory lacks an issue date — see escalate rule 3).

Role	Laptop	Monitor	Ergonomic chair
Engineering	1 every 3 years	up to 2 every 2 years	1 every 5 years
Sales	1 every 4 years	1 every 3 years	1 every 5 years
Operations	1 every 4 years	1 every 3 years	1 every 5 years
Management	1 every 2 years	2 every 2 years	1 every 5 years

Policy interpretation rules

- Within limit means: for the requested item, either the employee has never been issued that item, or enough full years have elapsed since the last issue date to meet the role's minimum interval, and (for monitors under Engineering/Management) current owned count is below the max quantity.

- Exceeds frequency means: the minimum interval has not elapsed (or quantity would exceed the role max).
- Year intervals use calendar years from issued_date to “today” at evaluation time (mock “today” may be fixed in fixtures for deterministic tests).
- ergonomic_chair is a standard catalog item with a simple default of 1 every 5 years for all four roles, so chair requests remain testable even though the official laptop/monitor tables do not list chairs.

Unknown roles (anything outside the four named roles) are a structured error from policy lookup — not approve, deny, or escalate.

3. Standard vs non-standard catalog

Standard items (auto-eligible for approve/deny against policy)

Item key	Notes
laptop	Primary compute device; frequency by role (table above).
monitor	Display; quantity + frequency by role.
ergonomic_chair	Seating; 1 every 5 years for all standard roles.

Non-standard items (always escalate)

Examples (non-exhaustive; any item not in the standard list is non-standard):

- Mechanical keyboard
- Ergonomic split keyboard
- Standing desk
- Docking station variants not on the standard list
- Any free-text item that does not normalize to laptop, monitor, or ergonomic_chair

Non-standard requests must not be approved or denied by automatic policy; they require human review.

4. Escalate rules (bounded)

The system escalates only when one of these three criteria holds. Escalation must invoke flag_for_human_review (employee id, request summary, reason) and return decision escalate.

1. Non-standard item — item_requested is not on the standard catalog (laptop / monitor / ergonomic_chair).
2. Early refresh with exceptional justification — request exceeds frequency, but reason credibly claims exceptional circumstances. Recognized signal phrases (case-insensitive substring match is sufficient for the lab): damaged, crushed, stolen, broken screen, client site (as in damaged on client site). Without such justification, an early request is a clear deny, not escalate.
3. Missing or conflicting tenure / issue-date data — employee record exists but inventory issue dates are missing, malformed, or conflict with tenure fields so eligibility cannot be computed deterministically. Do not guess timelines; escalate.

Borderline timing without exceptional justification is deny, not escalate. Contractors, cost caps, and

manager overrides are out of scope for this lab.

5. Decision outcomes

Every evaluated request ends in exactly one of:

Outcome	When
approve	Item is standard; employee found; role known; within quantity/frequency limits; tenure/issue data sufficient.
deny	Item is standard; employee found; role known; exceeds frequency (or would exceed quantity) and reason does not contain exceptional justification.
escalate	Any of the three bounded escalate rules; after calling <code>flag_for_human_review</code> .

Supporting structured results (not final outcomes) include:

- Employee not found / invalid id → structured error (tests & tools); agent must not invent an approve.
- Unknown role on record → structured error from `get_policy_limits`.

Eligibility helpers may expose flags such as `eligible`, `decision_hint` (approve / deny / escalate), `exceeds_frequency`, `last_issued`, `next_eligible_date`, and human-readable reasons. They do not inspect free-text justification. The authoritative final outcome comes from `evaluate_request(employee_id, item, reason)`, which applies exceptional-justification phrases on top of eligibility and returns `{decision, rule}`.

6. Mock employee fixtures (seed for scenarios & tests)

Reference “today” for age calculations in design and tests: 2026-10-02.

ID	Role	Inventory / history	Purpose
EMP-101	Engineering	Laptop issued 2022-09-01 (~4 years ago)	Clear approve — Engineering laptop ≥ 3 years; standard replacement reason.
EMP-102	Sales	Laptop issued 2026-04-01 (~6 months ago)	Clear deny — Sales laptop interval is 4 years; reason has no damage/exception claim.
EMP-103	Engineering	Laptop issued 2025-10-01 (~1 year ago)	Escalation A — early vs 3-year Engineering rule, but reason claims crushed / stolen / broken screen → must <code>flag_for_human_review</code> → escalate.
EMP-104	Management	Monitor x1 issued 2024-10-01; chair none	Supporting monitor/chair cases (within Management limits when applicable).
EMP-105	Engineering	Laptop present but <code>issued_date</code> missing/conflicting with <code>tenure</code>	Tenure-data escalate (optional extra / unit tests).
EMP-999	—	Absent from DB	Not found — supporting edge in unit tests.

Target graded scenarios (Phase 6 evidence)

1. Clear approval — EMP-101 requests laptop with a normal wear/performance reason → `evaluate_request` → approve (within_policy). Must not escalate.
2. Clear denial — EMP-102 requests laptop replacement after ~6 months with no exceptional justification → `evaluate_request` → deny (exceeds_frequency). Must not escalate.

3. Escalation A — EMP-103 early laptop + exceptional reason → evaluate_request → escalate (early_refresh_exceptional) → call flag_for_human_review → escalate.
4. Escalation B — Any seeded employee (e.g. EMP-101 or EMP-104) requests non-standard gear (e.g. ergonomic split keyboard or standing desk) → evaluate_request → escalate (non_standard_item) → call flag_for_human_review → escalate.

7. Escalation queue (side effect)

flag_for_human_review(employee_id, request, reason) appends a record:

```
[json]
{
  "timestamp": "<ISO-8601 UTC>",
  "employee_id": "EMP-103",
  "request": "laptop",
  "reason": "<human-readable escalation reason>"
}
```

Storage for the lab: in-memory list for the process lifetime, plus optional append to data/escalations.jsonl. Return a confirmation dict including the written fields.

8. Plain-function + MCP surface (implemented)

These functions encode this document in business_logic.py and are exposed as MCP tools in server.py.

Function	Responsibility
get_employee_info(employee_id)	Role, tenure, hardware inventory; structured error if missing/invalid.
get_policy_limits(role)	Per-item max quantity + min years; error if unknown role.
check_request_eligibility(employee_id, item)	Compose employee + policy; eligibility hints only (no free-text reason).
evaluate_request(employee_id, item, reason)	Authoritative {decision, rule} including exceptional-justification matching.
flag_for_human_review(employee_id, request, reason)	Queue append + confirmation (required whenever decision is escalate).

Stable evaluate_request.rule ids include: within_policy, exceeds_frequency, early_refresh_exceptional, non_standard_item, missing_tenure.

Done criteria

A grader can reconstruct approve / deny / escalate outcomes for the four scenarios from this document alone, and the agent must copy evaluate_request.decision rather than inventing policy.

2. Minimal one-tool + full-server MCP handshake

Suggested Approach step 3 — trivial ping server/client before business logic:

Minimal one-tool MCP handshake

```
Connecting to Minimal-Ping-Server over stdio...
=== Handshake (initialize) ===
Protocol: 2025-11-25
Server:   Minimal-Ping-Server
Registered tools: ['ping']
ping() -> pong
Minimal one-tool server/client connection confirmed.
```

Minimal one-tool MCP handshake

[text]

```
Connecting to Minimal-Ping-Server over stdio...
=== Handshake (initialize) ===
Protocol: 2025-11-25
Server:   Minimal-Ping-Server
Registered tools: ['ping']
ping() -> pong
Minimal one-tool server/client connection confirmed.
```

Full equipment server handshake (python test_client.py):

Full server MCP handshake (test_client.py)

```
Connecting to IT-Equipment-Server over stdio...

=== Handshake (initialize) ===
Protocol: 2025-11-25
Server:   IT-Equipment-Server
Version:  1.30.0

=== Tools discovered (list_tools) ===
- get_employee_info: Look up an employee record from the mock IT equipment database.
- find_employee: Resolve EMP-### from a person's name and optional department/role.
- get_policy_limits: Fetch per-item quantity and frequency limits for a job role.
- check_request_eligibility: Evaluate whether an employee may receive a requested equipment item.
- evaluate_request: Apply the full policy procedure and return {decision, rule}.
- flag_for_human_review: Queue an equipment request for human review (required for all escalations).

Registered tools OK: ['check_request_eligibility', 'evaluate_request', 'find_employee', 'flag_for_human_review', 'get_employee_info', 'get_policy_limits']

=== call_tool("get_employee_info", {"employee_id": "EMP-101"}) ===
{
  "ok": true,
  "employee_id": "EMP-101",
  "name": "Alex Rivera",
  "role": "Engineering",
  "hire_date": "2019-03-15",
  "inventory": [
    {
      "item": "laptop",
      "issued_date": "2022-09-01",
      "asset_tag": "LT-101"
    }
  ]
}

MCP client smoke test complete.
```

Phase 3 MCP handshake

3. Full MCP server implementation

[python]

```
'''
This script starts the IT-Equipment-Server over stdio.
1. Initializes the connection
2. Lists the tools
3. Calls the get_employee_info() tool
'''
```

4. Prints the result

```
'''

from __future__ import annotations

from typing import Any

from mcp.server.fastmcp import FastMCP

import business_logic as bl

mcp = FastMCP("IT-Equipment-Server")


@mcp.tool()
def get_employee_info(employee_id: str) -> dict[str, Any]:
    """Look up an employee record from the mock IT equipment database.

    Problem this solves: agents need the authoritative role, tenure, and hardware
    inventory before applying replacement policy. Free-text role claims are not trusted.

    When to call: at the start of an equipment request once you have an employee_id
    (pattern EMP-###), or whenever inventory/role must be re-checked.

    Args:
        employee_id: Employee identifier such as "EMP-101".

    Returns:
        Success: {ok, employee_id, name, role, hire_date, inventory}.
        Failure: {ok: false, error, message, ...} for invalid or missing ids.
    """
    return bl.get_employee_info(employee_id)


@mcp.tool()
def find_employee(name: str, role: str = "") -> dict[str, Any]:
    """Resolve EMP-### from a person's name and optional department/role.

    Problem this solves: users often identify themselves by name and department
    instead of employee_id. Other tools require EMP-### and must not receive a
    display name as employee_id.

    When to call: at the start when the request has a name (e.g. "Alex Rivera")
    and/or department but no EMP-###. Then call get_employee_info with the
    returned employee_id. Claimed department only disambiguates; record role wins.

    Args:
        name: Full or partial employee name, e.g. "Alex Rivera".
        role: Optional claimed department/role (Engineering, Sales, Operations,
            Management). Pass "" if unknown.

    Returns:
        Success: {ok, employee_id, name, role, matched_by, note}.
        Failure: {ok: false, error, message, candidates?} if missing/ambiguous.
    """
    return bl.find_employee(name, role or None)


@mcp.tool()
def get_policy_limits(role: str) -> dict[str, Any]:
    """Fetch per-item quantity and frequency limits for a job role.

    Problem this solves: eligibility depends on role-specific max quantity and
    minimum years between issuances for laptop, monitor, and ergonomic_chair.
```

When to call: after you know the employee's authoritative role from `get_employee_info`, and before or while interpreting `check_request_eligibility`.
Do not invent limits from memory – always read them from this tool.

Args:

role: One of Engineering, Sales, Operations, Management
(alias "Manager" is normalized to Management).

Returns:

Success: {ok, role, limits: {item: {max_qty, min_years}}, standard_items}.

Failure: {ok: false, error, message} for unknown or empty roles.

"""

return bl.get_policy_limits(role)

@mcp.tool()

def check_request_eligibility(employee_id: str, item: str) -> dict[str, Any]:

"""Evaluate whether an employee may receive a requested equipment item.

Problem this solves: composes employee inventory with role policy into a deterministic eligibility result (approve / deny / escalate hints) without duplicating policy math in the agent. Does NOT inspect free-text reason.

When to call: optional diagnostic after resolving `employee_id` and `item`.
Prefer `evaluate_request` for the authoritative approve/deny/escalate decision (it applies exceptional-justification phrases on top of this eligibility).

Args:

employee_id: Employee identifier such as "EMP-101".

item: Requested item key (e.g. "laptop", "monitor", "ergonomic_chair",
or a non-standard name like "standing_desk").

Returns:

Success shape includes ok, eligible, decision_hint, exceeds_frequency,
last_issued, next_eligible_date, owned_count, reasons, and often policy.

Failure: structured error if the employee id is invalid or not found.

"""

return bl.check_request_eligibility(employee_id, item)

@mcp.tool()

def evaluate_request(employee_id: str, item: str, reason: str) -> dict[str, Any]:

"""Apply the full policy procedure and return {decision, rule}.

Problem this solves: eligibility math alone cannot decide early-refresh cases;
free-text exceptional justification must be applied deterministically. This
tool is the authoritative decision – do not invent approve/deny/escalate.

When to call: after you know `employee_id` and the requested item, with the
requester's free-text reason. Then:

- decision=approve or deny → draft Final Decision with that decision
- decision=escalate → call `flag_for_human_review`, then Final Decision escalate

Args:

employee_id: Employee identifier such as "EMP-101".

item: Requested item key (e.g. "laptop", "standing_desk").

reason: Free-text justification from the requester.

Returns:

Success: {ok, decision, rule, eligibility, reasons,
has_exceptional_justification, next_action?}.

Failure: ok=false with rule/error when lookup fails or reason is empty.

"""

return bl.evaluate_request(employee_id, item, reason)

```
@mcp.tool()
def flag_for_human_review(employee_id: str, request: str, reason: str) -> dict[str, Any]:
    """Queue an equipment request for human review (required for all escalations).

    Problem this solves: bounded ambiguity must not be resolved by guessing –
    non-standard gear, early refresh with exceptional justification, or missing /
    conflicting tenure data need a human. This tool records the escalation.

    When to call: when evaluate_request returns decision=escalate (or next_action
    is flag_for_human_review). Call before Final Decision: escalate. Do not call
    for clear approve or clear deny paths.

    Args:
        employee_id: Employee identifier such as "EMP-103".
        request: Short request summary (usually the item key, e.g. "laptop").
        reason: Why human review is required (policy conflict, damage claim, etc.).

    Returns:
        Success: {ok, flagged, queue_size, entry, message}.
        entry is {timestamp, employee_id, request, reason}.
        Failure: {ok: false, error, message} if required fields are empty.
    """
    return bl.flag_for_human_review(employee_id, request, reason)

if __name__ == "__main__":
    mcp.run(transport="stdio")
```

4. Unit test suite and passing terminal logs

`test\_business\_logic.py`

```
[python]
'''
This script contains the unit tests for the business logic.
1. get_employee_info()
2. get_policy_limits()
3. check_request_eligibility()
4. evaluate_request()
5. flag_for_human_review().
'''

from __future__ import annotations

import json
from datetime import datetime

import pytest

import business_logic as bl

@pytest.fixture(autouse=True)
def _clean_escalations():
    """Clear the escalation queue and truncate the file."""
    bl.clear_escalation_queue(truncate_file=True)
    yield
    bl.clear_escalation_queue(truncate_file=True)
```

```

# --- get_employee_info ---

def test_get_employee_info_valid_engineering():
    """EMP-101: valid engineering employee."""
    result = bl.get_employee_info("EMP-101")
    assert result["ok"] is True
    assert result["employee_id"] == "EMP-101"
    assert result["role"] == "Engineering"
    assert result["hire_date"] == "2019-03-15"
    assert any(i["item"] == "laptop" for i in result["inventory"])

def test_get_employee_info_nonexistent():
    """EMP-999: nonexistent employee."""
    result = bl.get_employee_info("EMP-999")
    assert result["ok"] is False
    assert result["error"] == "employee_not_found"

def test_get_employee_info_invalid_id():
    """Invalid employee ID: empty string."""
    result = bl.get_employee_info("")
    assert result["ok"] is False
    assert result["error"] == "invalid_employee_id"

    result = bl.get_employee_info("USER-1")
    assert result["ok"] is False
    assert result["error"] == "invalid_employee_id"

def test_find_employee_by_name_and_role():
    result = bl.find_employee("Alex Rivera", "Engineering")
    assert result["ok"] is True
    assert result["employee_id"] == "EMP-101"
    assert result["role"] == "Engineering"

def test_find_employee_name_only():
    result = bl.find_employee("Jordan Lee")
    assert result["ok"] is True
    assert result["employee_id"] == "EMP-102"

def test_find_employee_not_found():
    result = bl.find_employee("Nobody Here")
    assert result["ok"] is False
    assert result["error"] == "employee_not_found"

# --- get_policy_limits ---

def test_get_policy_limits_engineering():
    """Engineering role policy limits."""
    result = bl.get_policy_limits("Engineering")
    assert result["ok"] is True
    assert result["limits"]["laptop"] == {"max_qty": 1, "min_years": 3}
    assert result["limits"]["monitor"] == {"max_qty": 2, "min_years": 2}
    assert result["limits"]["ergonomic_chair"] == {"max_qty": 1, "min_years": 5}

def test_get_policy_limits_unknown_role():
    """Unknown role: Intern."""
    result = bl.get_policy_limits("Intern")

```

```

assert result["ok"] is False
assert result["error"] == "unknown_role"

# --- check_request_eligibility: approve / deny / escalate hints ---

def test_eligible_approve_engineering_laptop_four_years():
    """EMP-101: laptop issued 2022-09-01 vs Engineering 3-year rule → approve."""
    result = bl.check_request_eligibility("EMP-101", "laptop")
    assert result["ok"] is True
    assert result["eligible"] is True
    assert result["decision_hint"] == "approve"
    assert result["exceeds_frequency"] is False
    assert result["last_issued"] == "2022-09-01"
    assert result["years_since_last"] >= 3

def test_deny_sales_laptop_six_months():
    """EMP-102: laptop ~6 months ago, Sales 4-year rule → deny / exceeds_frequency."""
    result = bl.check_request_eligibility("EMP-102", "laptop")
    assert result["ok"] is True
    assert result["eligible"] is False
    assert result["decision_hint"] == "deny"
    assert result["exceeds_frequency"] is True
    assert result["last_issued"] == "2026-04-01"
    assert result["next_eligible_date"] == "2030-04-01"

def test_exceeds_frequency_early_engineering_for_agent_escalation():
    """EMP-103 early laptop: exceeds_frequency so agent can escalate on damage claim."""
    result = bl.check_request_eligibility("EMP-103", "laptop")
    assert result["ok"] is True
    assert result["eligible"] is False
    assert result["exceeds_frequency"] is True
    assert result["decision_hint"] == "deny"
    assert result["last_issued"] == "2025-10-01"

def test_non_standard_item_escalate_hint():
    """EMP-101: standing desk → escalate, do not guess."""
    result = bl.check_request_eligibility("EMP-101", "standing_desk")
    assert result["ok"] is True
    assert result["eligible"] is False
    assert result["decision_hint"] == "escalate"
    assert result["exceeds_frequency"] is False
    reason = result["reasons"][0].lower()
    assert "standard catalog" in reason or "not on the standard" in reason

def test_missing_tenure_data_escalate():
    """EMP-105: laptop with null issued_date → escalate, do not guess."""
    result = bl.check_request_eligibility("EMP-105", "laptop")
    assert result["ok"] is True
    assert result["eligible"] is False
    assert result["decision_hint"] == "escalate"
    reason = result["reasons"][0].lower()
    assert "issued_date" in reason or "missing" in reason

def test_eligibility_invalid_employee():
    """EMP-999: nonexistent employee."""
    result = bl.check_request_eligibility("EMP-999", "laptop")
    assert result["ok"] is False
    assert result["error"] == "employee_not_found"

```

```

def test_management_chair_never_owned_approve():
    """EMP-104: ergonomic chair never owned → approve."""
    result = bl.check_request_eligibility("EMP-104", "ergonomic_chair")
    assert result["ok"] is True
    assert result["eligible"] is True
    assert result["decision_hint"] == "approve"

def test_management_second_monitor_under_max_qty():
    """EMP-104 has 1 monitor; Management max is 2 → approve additional."""
    result = bl.check_request_eligibility("EMP-104", "monitor")
    assert result["ok"] is True
    assert result["eligible"] is True
    assert result["decision_hint"] == "approve"
    assert result["owned_count"] == 1

# --- evaluate_request: authoritative approve / deny / escalate ---

def test_evaluate_approve_engineering_laptop():
    """Graded approve: EMP-101 laptop within 3-year Engineering rule."""
    result = bl.evaluate_request(
        "EMP-101",
        "laptop",
        "My laptop is 4 years old and slow; need a replacement.",
    )
    assert result["ok"] is True
    assert result["decision"] == "approve"
    assert result["rule"] == "within_policy"
    assert result["eligibility"]["eligible"] is True

def test_evaluate_deny_sales_laptop_no_exception():
    """Graded deny: EMP-102 early laptop with only performance complaints."""
    result = bl.evaluate_request(
        "EMP-102",
        "laptop",
        "Laptop is freezing and old; please replace it.",
    )
    assert result["ok"] is True
    assert result["decision"] == "deny"
    assert result["rule"] == "exceeds_frequency"
    assert result["has_exceptional_justification"] is False

def test_evaluate_escalate_early_refresh_exceptional():
    """Graded escalate A: EMP-103 early laptop + crushed/stolen justification."""
    result = bl.evaluate_request(
        "EMP-103",
        "laptop",
        "Screen was crushed on a client site / stolen bag – broken screen.",
    )
    assert result["ok"] is True
    assert result["decision"] == "escalate"
    assert result["rule"] == "early_refresh_exceptional"
    assert result["has_exceptional_justification"] is True
    assert result["next_action"] == "flag_for_human_review"
    # Eligibility alone still says deny; evaluate_request upgrades on phrases.
    assert result["eligibility"]["decision_hint"] == "deny"
    assert result["eligibility"]["exceeds_frequency"] is True

```

```

def test_evaluate_escalate_non_standard_item():
    """Graded escalate B: non-standard catalog item."""
    result = bl.evaluate_request(
        "EMP-101",
        "ergonomic_split_keyboard",
        "Need a split keyboard for RSI.",
    )
    assert result["ok"] is True
    assert result["decision"] == "escalate"
    assert result["rule"] == "non_standard_item"
    assert result["next_action"] == "flag_for_human_review"

def test_evaluate_escalate_missing_tenure():
    """EMP-105 missing issued_date → escalate / missing_tenure."""
    result = bl.evaluate_request(
        "EMP-105",
        "laptop",
        "Need a laptop refresh; unsure when mine was issued.",
    )
    assert result["ok"] is True
    assert result["decision"] == "escalate"
    assert result["rule"] == "missing_tenure"

def test_evaluate_early_without_exception_stays_deny():
    """EMP-103 early laptop without exceptional phrases stays deny."""
    result = bl.evaluate_request(
        "EMP-103",
        "laptop",
        "It feels slow and I want a newer model.",
    )
    assert result["ok"] is True
    assert result["decision"] == "deny"
    assert result["rule"] == "exceeds_frequency"

def test_evaluate_unknown_employee():
    result = bl.evaluate_request("EMP-999", "laptop", "Need a laptop.")
    assert result["ok"] is False
    assert result["decision"] == "escalate"
    assert result["rule"] == "employee_not_found"

def test_evaluate_requires_nonempty_reason():
    result = bl.evaluate_request("EMP-101", "laptop", "")
    assert result["ok"] is False
    assert result["error"] == "invalid_reason"

def test_has_exceptional_justification_phrases():
    assert bl.has_exceptional_justification("crushed on client site") is True
    assert bl.has_exceptional_justification("just slow") is False
    assert bl.has_exceptional_justification(None) is False

# --- flag_for_human_review side effects ---

def test_flag_for_human_review_queue_and_timestamp():
    """EMP-103: laptop → flag for human review."""
    result = bl.flag_for_human_review(
        "EMP-103",
        "laptop",
        "Screen crushed on client site; early replacement needed",
    )

```

```

)
assert result["ok"] is True
assert result["flagged"] is True
assert result["queue_size"] == 1

entry = result["entry"]
assert entry["employee_id"] == "EMP-103"
assert entry["request"] == "laptop"
assert "crushed" in entry["reason"].lower()
# ISO-8601 parseable timestamp
datetime.fromisoformat(entry["timestamp"])

assert len(bl.ESCALATION_QUEUE) == 1
assert bl.ESCALATION_QUEUE[0]["employee_id"] == "EMP-103"

# JSONL append side effect
assert bl.ESCALATIONS_PATH.exists()
raw = bl.ESCALATIONS_PATH.read_text(encoding="utf-8")
lines = [ln for ln in raw.splitlines() if ln.strip()]
assert len(lines) == 1
saved = json.loads(lines[0])
assert saved["employee_id"] == "EMP-103"
assert saved["reason"] == entry["reason"]

def test_flag_for_human_review_requires_fields():
    """Empty fields: employee_id, request, reason."""
    assert bl.flag_for_human_review("", "laptop", "reason")["ok"] is False
    assert bl.flag_for_human_review("EMP-101", "", "reason")["ok"] is False
    assert bl.flag_for_human_review("EMP-101", "laptop", "")["ok"] is False
    assert bl.flag_for_human_review("EMP-101", "laptop", None)["ok"] is False

```

pytest terminal log

[text]

```

===== test session starts =====
platform linux -- Python 3.12.14, pytest-9.1.1, pluggy-1.6.0 -- /workspace/mcp/w7_mcp/.venv/bin/python
cachedir: .pytest_cache
rootdir: /workspace/mcp/w7_mcp
configfile: pyproject.toml
plugins: anyio-4.15.1
collecting ... collected 27 items

test_business_logic.py::test_get_employee_info_valid_engineering PASSED [ 3%]
test_business_logic.py::test_get_employee_info_nonexistent PASSED [ 7%]
test_business_logic.py::test_get_employee_info_invalid_id PASSED [ 11%]
test_business_logic.py::test_find_employee_by_name_and_role PASSED [ 14%]
test_business_logic.py::test_find_employee_name_only PASSED [ 18%]
test_business_logic.py::test_find_employee_not_found PASSED [ 22%]
test_business_logic.py::test_get_policy_limits_engineering PASSED [ 25%]
test_business_logic.py::test_get_policy_limits_unknown_role PASSED [ 29%]
test_business_logic.py::test_eligible_approve_engineering_laptop_four_years PASSED [ 33%]
test_business_logic.py::test_deny_sales_laptop_six_months PASSED [ 37%]
test_business_logic.py::test_exceeds_frequency_early_engineering_for_agent_escalation PASSED [ 40%]
test_business_logic.py::test_non_standard_item_escalate_hint PASSED [ 44%]
test_business_logic.py::test_missing_tenure_data_escalate PASSED [ 48%]
test_business_logic.py::test_eligibility_invalid_employee PASSED [ 51%]
test_business_logic.py::test_management_chair_never_owned_approve PASSED [ 55%]
test_business_logic.py::test_management_second_monitor_under_max_qty PASSED [ 59%]
test_business_logic.py::test_evaluate_approve_engineering_laptop PASSED [ 62%]
test_business_logic.py::test_evaluate_deny_sales_laptop_no_exception PASSED [ 66%]
test_business_logic.py::test_evaluate_escalate_early_refresh_exceptional PASSED [ 70%]
test_business_logic.py::test_evaluate_escalate_non_standard_item PASSED [ 74%]

```

```

test_business_logic.py::test_evaluate_escalate_missing_tenure PASSED      [ 77%]
test_business_logic.py::test_evaluate_early_without_exception_stays_deny PASSED [ 81%]
test_business_logic.py::test_evaluate_unknown_employee PASSED           [ 85%]
test_business_logic.py::test_evaluate_requires_nonempty_reason PASSED    [ 88%]
test_business_logic.py::test_has_exceptional_justification_phrases PASSED [ 92%]
test_business_logic.py::test_flag_for_human_review_queue_and_timestamp PASSED [ 96%]
test_business_logic.py::test_flag_for_human_review_requires_fields PASSED [100%]

===== 27 passed in 0.01s =====

```

5. Agent code (MCP client connection + ReAct loop)

`agent.py`

```

[python]
"""ReAct agent over the IT Equipment MCP server (Ollama + stdio)."""

from __future__ import annotations

import argparse
import asyncio
import json
import os
import re
import sys
from collections.abc import Awaitable, Callable
from datetime import datetime, timezone
from pathlib import Path
from typing import Any, Literal

import jsonschema
from dotenv import load_dotenv
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client
from pydantic import BaseModel, ConfigDict, Field, ValidationError, field_validator, model_validator

import llm
from prompts import REFLECTION_PROMPT, build_system_prompt

load_dotenv()

ROOT = Path(__file__).resolve().parent
TRACES_DIR = ROOT / "traces"

DecisionName = Literal["approve", "deny", "escalate"]
ReflectionAction = Literal["PROCEED", "DENY", "ESCALATE"]
Complete = Callable[[str], str]
CallTool = Callable[[str, dict[str, Any]], Awaitable[str]]
Emit = Callable[[str], None]

REPEAT_NOTICE = (
    "Already called with these arguments. "
    "Choose a different action, or finish if the trace is enough."
)

_GUARD_PREFIXES = (
    "Schema rejected:",
    "Unknown tool:",
    "Reply was not a JSON object.",
    "evaluate_request is not in the trace.",
    "evaluate_request.decision is ",
    "escalate requires flag_for_human_review",

```

```

    "Already called with these arguments.",
    "Blocked find_employee:",
)
_NEXT_MARKERS = (
    "Thought:",
    "Action:",
    "Final Decision:",
    "Reason:",
    "Draft ",
    "[HOST]",
    "==",
    "Reflection:",
    "Schema rejected:",
    "Request:",
    "[ROUTE]",
    "accept:",
)

FINAL_RE = re.compile(
    r"Final\s*Decision\s*:\s*(approve|deny|escalate)\b",
    re.IGNORECASE,
)

REASON_RE = re.compile(
    r"Reason\s*:\s*(.+)",
    re.IGNORECASE | re.DOTALL,
)

EMP_ID_RE = re.compile(r"\bEMP-\d+\b", re.IGNORECASE)

def _max_steps() -> int:
    return int(os.getenv("AGENT_MAX_STEPS", "8"))

class AgentStep(BaseModel):
    """One model turn: a tool call, or finish with a copied decision."""

    model_config = ConfigDict(extra="ignore")
    thought: str = Field(min_length=1)
    action: str = Field(min_length=1)
    action_input: dict[str, Any] = Field(default_factory=dict)
    decision: DecisionName | None = None
    reason: str | None = None

    @field_validator("action_input", mode="before")
    @classmethod
    def parse_action_input(cls, value: object) -> object:
        if value is None:
            return {}
        if isinstance(value, str):
            try:
                loaded = json.loads(value)
            except json.JSONDecodeError:
                return value
            return loaded
        return value

    @model_validator(mode="after")
    def finish_fields(self) -> AgentStep:
        if self.action == "finish":
            if self.decision is None:
                raise ValueError("decision is required when action is finish")
            if not (self.reason and self.reason.strip()):
                raise ValueError("reason is required when action is finish")
        elif self.decision is not None:
            raise ValueError("decision must be null unless action is finish")

```

```

        return self

class ReflectionReply(BaseModel):
    """Checkpoint after the draft: proceed, deny, or escalate."""

    model_config = ConfigDict(extra="ignore")
    is_valid: bool
    critique: str = Field(min_length=1)
    action: ReflectionAction

    @field_validator("is_valid", mode="before")
    @classmethod
    def parse_is_valid(cls, value: object) -> object:
        if isinstance(value, str):
            lowered = value.strip().lower()
            if lowered == "true":
                return True
            if lowered == "false":
                return False
        return value

    @field_validator("action", mode="before")
    @classmethod
    def parse_action(cls, value: object) -> object:
        if isinstance(value, str):
            return value.strip().upper()
        return value

class EvaluateResult(BaseModel):
    """Authoritative payload from evaluate_request. Extra fields are kept."""

    model_config = ConfigDict(extra="allow")
    decision: DecisionName
    rule: str = ""
    ok: bool | None = None

class FlagResult(BaseModel):
    """Successful flag_for_human_review payload."""

    model_config = ConfigDict(extra="allow")
    ok: bool
    flagged: bool

def extract_explicit_employee_id(text: str) -> str | None:
    """Return the first EMP-### in the prompt, or None if absent."""
    match = EMP_ID_RE.search(text or "")
    if not match:
        return None
    raw = match.group(0)
    return "EMP-" + raw.split("-", 1)[1]

def _schema_error(exc: ValidationError) -> str:
    parts: list[str] = []
    for err in exc.errors():
        loc = ".".join(str(item) for item in err["loc"]) or "body"
        parts.append(loc + ": " + err["msg"])
    return "Schema rejected: " + "; ".join(parts)

def _load_object(raw: str) -> dict[str, Any] | None:

```

```

text = raw.strip()
if not text:
    return None
try:
    data = json.loads(text)
except json.JSONDecodeError:
    start = text.find("{")
    end = text.rfind("}")
    if start < 0 or end <= start:
        return None
    try:
        data = json.loads(text[start : end + 1])
    except json.JSONDecodeError:
        return None
if not isinstance(data, dict):
    return None
return data

def _without_tool_decision(raw: str) -> str:
    """Drop a decision that arrived on a tool call. finish keeps its decision."""
    data = _load_object(raw)
    if not isinstance(data, dict) or data.get("action") == "finish" or data.get("decision") is None:
        return raw
    cleaned = dict(data)
    cleaned["decision"] = None
    return json.dumps(cleaned)

def parse_step(raw: str) -> AgentStep | str:
    """Validate one model reply. A string return is an observation for the model."""
    data = _load_object(_without_tool_decision(raw))
    if data is None:
        return "Reply was not a JSON object."
    try:
        return AgentStep.model_validate(data)
    except ValidationError as exc:
        return _schema_error(exc)

def _coerce_reflection(data: dict[str, Any]) -> dict[str, Any] | None:
    """Accept the checkpoint object, or the older decision/reason object."""
    if "action" in data and "is_valid" in data:
        critique = data.get("critique") or data.get("reflection") or data.get("reason")
        if not critique:
            return None
        return {
            "is_valid": data["is_valid"],
            "critique": str(critique),
            "action": data["action"],
        }
    decision = data.get("decision")
    critique = str(data.get("critique") or data.get("reflection") or data.get("reason") or "")
    critique = critique.strip()
    if not critique or decision not in {"approve", "deny", "escalate", "undecided"}:
        return None
    if decision == "undecided":
        return {"is_valid": False, "critique": critique, "action": "ESCALATE"}
    if decision == "deny":
        return {"is_valid": False, "critique": critique, "action": "DENY"}
    if decision == "escalate":
        return {"is_valid": True, "critique": critique, "action": "ESCALATE"}
    return {"is_valid": True, "critique": critique, "action": "PROCEED"}

```

```

def parse_reflection(text: str) -> dict[str, Any]:
    """Parse a reflection checkpoint. A bad reply leaves action unset."""
    data = _load_object(text)
    if data is None:
        return {
            "reflection": "Reply was not a JSON object.",
            "is_valid": None,
            "critique": "",
            "action": None,
            "raw": text,
        }
    coerced = _coerce_reflection(data)
    if coerced is None:
        return {
            "reflection": "Reply was not a reflection checkpoint.",
            "is_valid": None,
            "critique": "",
            "action": None,
            "raw": text,
        }
    try:
        reply = ReflectionReply.model_validate(coerced)
    except ValidationError as exc:
        return {
            "reflection": _schema_error(exc),
            "is_valid": None,
            "critique": "",
            "action": None,
            "raw": text,
        }
    return {
        "reflection": reply.critique,
        "is_valid": reply.is_valid,
        "critique": reply.critique,
        "action": reply.action,
        "raw": text,
    }

def reflection_checkpoint(
    *,
    draft: str | None,
    policy: str | None,
    action: str | None,
) -> dict[str, Any]:
    """Choose the saved decision after reflection.

    PROCEED keeps a draft that matches the tool. DENY sticks only when the
    tool said deny. ESCALATE sticks when the tool said escalate, or when no
    tool result exists. Any other disagreement snaps back to the tool.
    """
    if action not in {"PROCEED", "DENY", "ESCALATE"}:
        return {"decision": draft, "flag": False, "label": "unparsed"}
    if action == "PROCEED" and (policy is None or draft == policy):
        return {"decision": draft, "flag": False, "label": "proceed"}
    if action == "DENY" and policy in {None, "deny"}:
        return {"decision": "deny", "flag": False, "label": "deny"}
    if action == "ESCALATE" and policy in {None, "escalate"}:
        return {"decision": "escalate", "flag": True, "label": "escalate"}
    return {
        "decision": policy or draft,
        "flag": policy == "escalate",
        "label": "lock",
    }

```

```

def parse_injected_draft(text: str) -> dict[str, Any]:
    """Read the reflection-demo draft. Accepts the legacy text shape or JSON."""
    final_m = FINAL_RE.search(text)
    if final_m:
        reason_m = REASON_RE.search(text)
        return {
            "kind": "final",
            "decision": final_m.group(1).lower(),
            "reason": (reason_m.group(1).strip() if reason_m else ""),
        }
    data = _load_object(text)
    if data and data.get("decision") in {"approve", "deny", "escalate"}:
        return {
            "kind": "final",
            "decision": str(data["decision"]),
            "reason": str(data.get("reason") or ""),
        }
    return {"kind": "invalid"}

def _tool_name(tool: Any) -> str:
    if isinstance(tool, dict):
        return str(tool.get("name") or "")
    return str(getattr(tool, "name", "") or "")

def _tool_schema(tool: Any) -> dict[str, Any]:
    if isinstance(tool, dict):
        schema = tool.get("inputSchema")
    else:
        schema = getattr(tool, "inputSchema", None)
    if isinstance(schema, dict):
        return schema
    return {}

def _json_schema_gap(schema: dict[str, Any], data: object) -> str | None:
    if not schema:
        return None
    validator = jsonschema.Draft202012Validator(schema)
    errors = sorted(validator.iter_errors(data), key=lambda err: list(err.absolute_path))
    if not errors:
        return None
    parts: list[str] = []
    for err in errors:
        loc = ".".join(str(part) for part in err.absolute_path) or "body"
        parts.append(loc + ": " + err.message)
    return "Schema rejected: " + "; ".join(parts)

def _validate_tool_result(action: str, text: str) -> str | None:
    """Check evaluate and flag payloads. Error objects (ok false) are left alone."""
    data = _load_object(text)
    if data is None:
        return "Schema rejected: Tool result was not a JSON object."
    if data.get("ok") is False:
        return None
    if action == "evaluate_request":
        model: type[BaseModel] = EvaluateResult
    elif action == "flag_for_human_review":
        model = FlagResult
    else:
        return None
    try:

```

```

        model.model_validate(data)
    except ValidationError as exc:
        return _schema_error(exc)
    return None

def _read_evaluate(text: str) -> EvaluateResult | None:
    data = _load_object(text)
    if not isinstance(data, dict) or "decision" not in data:
        return None
    try:
        return EvaluateResult.model_validate(data)
    except ValidationError:
        return None

def _flag_succeeded(text: str) -> bool:
    data = _load_object(text)
    if not isinstance(data, dict) or data.get("ok") is False:
        return False
    try:
        return FlagResult.model_validate(data).flagged is True
    except ValidationError:
        return False

def _is_guard(body: str) -> bool:
    return body.startswith(_GUARD_PREFIXES)

def iter_actions(lines: list[str]) -> list[tuple[str, str]]:
    """Pair each Action line with the observation body that follows it."""
    pairs: list[tuple[str, str]] = []
    index = 0
    while index < len(lines):
        line = lines[index]
        if not line.startswith("Action: "):
            index += 1
            continue
        rest = line[len("Action: ") :]
        name = rest.split(" ", 1)[0]
        body_lines: list[str] = []
        if index + 1 < len(lines) and lines[index + 1].startswith("Observation:"):
            first = lines[index + 1][len("Observation:") :].strip()
            if first:
                body_lines.append(first)
            cursor = index + 2
            while cursor < len(lines) and not lines[cursor].startswith(_NEXT_MARKERS):
                body_lines.append(lines[cursor])
                cursor += 1
            pairs.append((name, "\n".join(body_lines).strip()))
        index += 1
    return pairs

def policy_from_lines(lines: list[str]) -> tuple[str | None, str | None]:
    """Latest evaluate_request decision and rule recorded in a trace."""
    found: EvaluateResult | None = None
    for name, body in iter_actions(lines):
        if name != "evaluate_request" or _is_guard(body):
            continue
        record = _read_evaluate(body)
        if record is not None:
            found = record
    if found is None:

```

```

        return None, None
    return found.decision, found.rule

def tools_called_from_lines(lines: list[str]) -> list[str]:
    """Tool names that were actually sent. Refusals and finish are omitted."""
    called: list[str] = []
    for name, body in iter_actions(lines):
        if name == "finish" or not body or _is_guard(body):
            continue
        called.append(name)
    return called

def last_decision(lines: list[str]) -> tuple[str | None, str]:
    """Last Final Decision line and the Reason line that follows it."""
    decision: str | None = None
    reason = ""
    for index, line in enumerate(lines):
        if not line.startswith("Final Decision:"):
            continue
        decision = line[len("Final Decision:") :].strip().lower()
        if index + 1 < len(lines) and lines[index + 1].startswith("Reason:"):
            reason = lines[index + 1][len("Reason:") :].strip()
    return decision, reason

def _call_key(action: str, arguments: dict[str, Any]) -> str:
    return action + " " + json.dumps(arguments, sort_keys=True)

def _finish_gap(
    policy: EvaluateResult | None,
    flagged: bool,
    decision: str,
) -> str | None:
    if policy is None:
        return "evaluate_request is not in the trace. Call it before finish."
    if decision != policy.decision:
        return (
            f"evaluate_request.decision is {policy.decision}. "
            f"Copy that decision. Finish with {policy.decision}."
        )
    if policy.decision == "escalate" and not flagged:
        return "escalate requires flag_for_human_review before finish."
    return None

def _flag_gap(policy: EvaluateResult | None) -> str | None:
    if policy is None:
        return (
            "evaluate_request is not in the trace. "
            "Call evaluate_request before flag_for_human_review."
        )
    if policy.decision in {"approve", "deny"}:
        return (
            f"evaluate_request.decision is {policy.decision}. "
            f"Do not call flag_for_human_review. Finish with {policy.decision}."
        )
    return None

def _step_prompt(
    request_text: str,
    trace: str,

```

```

    steps_remaining: int,
    completed: list[str],
    explicit_employee_id: str | None,
) -> str:
    finished = ""
    if completed:
        finished = (
            "Calls that already returned an observation:\n"
            + "\n".join(completed)
            + "\nDo not repeat them.\n"
        )
    hint = ""
    if explicit_employee_id:
        hint = (
            f"[HOST HINT] Explicit employee_id {explicit_employee_id} is present. "
            "Use that id. Do not call find_employee.\n"
        )
    return (
        "Reply with one JSON object and nothing else.\n"
        'Keys are "thought", "action", "action_input", "decision", and "reason".\n'
        "thought is your reasoning for this turn: what you know, what is missing, "
        "and why this action follows.\n"
        "action is finish, or a tool name.\n"
        "action_input is the tool arguments, or {} when action is finish.\n"
        'decision is "approve", "deny", or "escalate" only when action is finish, otherwise null.\n'
        "reason is required when action is finish, otherwise null.\n"
        "Decide the next action from the request and the trace. "
        "Call a tool when a fact you need is still missing. "
        "Finish when evaluate_request supports the decision. "
        "Copy that tool's decision. "
        "Call flag_for_human_review only after that tool returns escalate, and before finish.\n"
        + hint
        + finished
        + "Steps remaining: "
        + str(steps_remaining)
        + ".\n\nRequest:\n"
        + request_text
        + "\n\nTrace so far:\n"
        + (trace or "(empty)")
        + "\n"
    )

def _observation_text(result: Any) -> str:
    parts: list[str] = []
    if getattr(result, "isError", False):
        parts.append("ERROR")
    for block in getattr(result, "content", []) or []:
        text = getattr(block, "text", None)
        if text is None:
            continue
        try:
            parts.append(json.dumps(json.loads(text), indent=2))
        except json.JSONDecodeError:
            parts.append(str(text))
    return "\n".join(parts) if parts else str(result)

async def run_react(
    request_text: str,
    tools: list[Any],
    complete: Complete,
    call_tool: CallTool,
    *,
    explicit_employee_id: str | None = None,

```

```

max_steps: int | None = None,
emit: Emit | None = None,
) -> list[str]:
    """TAO loop with no Ollama and no MCP session.

    ``complete`` returns one JSON step. ``call_tool`` performs a tool call the
    host already accepted. Illegal flag and finish moves become observations.
    """
    limit = _max_steps() if max_steps is None else max_steps
    lines: list[str] = []
    by_name = {_tool_name(tool): tool for tool in tools}
    seen: dict[str, str] = {}
    policy: EvaluateResult | None = None
    policy_args: dict[str, Any] = {}
    flagged = False

    def add(text: str) -> None:
        for line in text.splitlines() or [""]:
            lines.append(line)
            if emit is not None:
                emit(line)

    add("Request: " + request_text)

    def remember_result(action: str, arguments: dict[str, Any], observation: str) -> str:
        nonlocal policy, policy_args, flagged
        gap = _validate_tool_result(action, observation)
        if action == "evaluate_request":
            record = _read_evaluate(observation)
            if record is not None:
                policy = record
                policy_args = arguments
        if action == "flag_for_human_review" and _flag_succeeded(observation):
            flagged = True
        if gap:
            observation = observation + "\n" + gap
        seen[_call_key(action, arguments)] = observation
        return observation

    for index in range(limit):
        parsed = parse_step(
            complete(
                _step_prompt(
                    request_text,
                    "\n".join(lines),
                    limit - index,
                    list(seen),
                    explicit_employee_id,
                )
            )
        )
        if isinstance(parsed, str):
            add("Observation: " + parsed)
            continue

        add("Thought: " + parsed.thought)
        if parsed.action == "finish":
            decision = parsed.decision
            reason = parsed.reason
            add("Action: finish {}")
            if decision is None or not (reason and reason.strip()):
                add(
                    "Observation: Schema rejected: "
                    "decision and reason are required when action is finish"
                )

```

```

        continue
    gap = _finish_gap(policy, flagged, decision)
    if gap is not None:
        add("Observation: " + gap)
        continue
    add("Final Decision: " + decision)
    add("Reason: " + reason.strip())
    return lines

if parsed.action not in by_name:
    add("Action: " + _call_key(parsed.action, parsed.action_input))
    add("Observation: Unknown tool: " + parsed.action)
    continue

arguments = dict(parsed.action_input)
if parsed.action == "find_employee" and explicit_employee_id:
    add("Action: " + _call_key(parsed.action, arguments))
    add(
        "Observation: Blocked find_employee: prompt already contains "
        f"{explicit_employee_id}. Do not resolve by name. "
        f'Next call get_employee_info with employee_id="{explicit_employee_id}", '
        "then evaluate_request with the same id."
    )
    continue

claimed_id = str(arguments.get("employee_id", "")).strip().upper()
if (
    explicit_employee_id
    and "employee_id" in arguments
    and claimed_id != explicit_employee_id.upper()
):
    arguments["employee_id"] = explicit_employee_id
    add(
        f"[HOST] Rewrote employee_id → {explicit_employee_id} "
        "(explicit id in prompt wins)."
    )

schema_gap = _json_schema_gap(_tool_schema(by_name[parsed.action]), arguments)
if schema_gap is not None:
    add("Action: " + _call_key(parsed.action, arguments))
    add("Observation: " + schema_gap)
    continue

if parsed.action == "flag_for_human_review":
    flag_gap = _flag_gap(policy)
    if flag_gap is not None:
        add("Action: " + _call_key(parsed.action, arguments))
        add("Observation: " + flag_gap)
        continue

key = _call_key(parsed.action, arguments)
if key in seen:
    add("Action: " + key)
    add("Observation: " + REPEAT_NOTICE)
    continue

add("Action: " + key)
observation = await call_tool(parsed.action, arguments)
observation = remember_result(parsed.action, arguments, observation)
add("Observation:\n" + observation)
if parsed.action == "evaluate_request" and policy is not None:
    add(
        f"[ROUTE] evaluate_request returned {policy.decision} "
        f"(rule={policy.rule})."
    )

```

```

if policy is not None and policy.decision == "escalate" and not flagged:
    flag_args = {
        "employee_id": str(policy_args.get("employee_id") or explicit_employee_id or ""),
        "request": str(policy_args.get("item") or policy_args.get("request") or "equipment"),
        "reason": f"evaluate_request rule={policy.rule}",
    }
    add("Thought: The step budget is spent. Store the escalation, then finish.")
    add("Action: " + _call_key("flag_for_human_review", flag_args))
    observation = remember_result(
        "flag_for_human_review",
        flag_args,
        await call_tool("flag_for_human_review", flag_args),
    )
    add("Observation:\n" + observation)
    add("Final Decision: escalate")
    add("Reason: " + (policy.rule or "escalate"))
    return lines

if policy is not None:
    add("Thought: The step budget is spent. Copy evaluate_request.")
    add("Final Decision: " + policy.decision)
    add("Reason: " + (policy.rule or policy.decision))
    return lines

add("Final Decision: escalate")
add(f"Reason: Exceeded max steps ({limit}); no evaluate_request result.")
return lines

def _slug(prompt: str) -> str:
    cleaned = re.sub(r"^[a-zA-Z0-9]+", "-", prompt.strip())[1:48].strip("-").lower()
    return cleaned or "request"

def _save_trace(
    prompt: str,
    lines: list[str],
    decision: str | None,
    *,
    path: Path | None = None,
    extra_header: dict[str, str] | None = None,
) -> Path:
    TRACES_DIR.mkdir(parents=True, exist_ok=True)
    if path is None:
        ts = datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%S")
        path = TRACES_DIR / f"{ts}_{_slug(prompt)}.txt"
    else:
        path.parent.mkdir(parents=True, exist_ok=True)

    header = [
        f"prompt: {prompt}",
        f"model: {llm.ollama_model()}",
        f"host: {llm.ollama_host()}",
        f"decision: {decision or 'none'}",
    ]
    if extra_header:
        for key, value in extra_header.items():
            header.append(f"{key}: {value}")
    header.append("")
    path.write_text("\n".join(header + lines) + "\n", encoding="utf-8")
    return path

def run_reflection(

```

```

*,
user_request: str,
draft: str,
observations: str,
) -> dict[str, Any]:
    """Review the draft before it is final. Evidence is the tool observations."""
    prompt = REFLECTION_PROMPT.format(
        user_request=user_request,
        draft=draft,
        observations=observations or "(no tool observations)",
    )
    raw = llm.chat(
        [
            {
                "role": "system",
                "content": (
                    "You are an IT compliance auditor. "
                    "The draft is not final. "
                    "PROCEED only when it matches evaluate_request. "
                    "DENY only when that tool said deny. "
                    "ESCALATE when that tool said escalate, or when you cannot tell. "
                    "Reply with one JSON object."
                ),
            },
            {"role": "user", "content": prompt},
        ],
        temperature=0.0,
        format="json",
    )
    return parse_reflection(raw)

def _action_arguments(lines: list[str], tool_name: str) -> dict[str, Any]:
    """Last JSON arguments recorded on an Action line for tool_name."""
    found: dict[str, Any] = {}
    prefix = f"Action: {tool_name} "
    for line in lines:
        if not line.startswith(prefix):
            continue
        data = _load_object(line[len(prefix):])
        if isinstance(data, dict):
            found = data
    return found

def _ensure_policy_rule(reason: str, policy_rule: str | None) -> str:
    """Keep evaluate_request.rule in the saved reason when the draft omitted it."""
    if not policy_rule or policy_rule in reason:
        return reason
    return f"{reason} [evaluate_request rule={policy_rule}].strip()"

def _observations_for_reflection(lines: list[str]) -> str:
    chunks: list[str] = []
    for name, body in iter_actions(lines):
        if name == "finish" or not body or _is_guard(body):
            continue
        chunks.append(f"{name}: {body}")
    return "\n\n".join(chunks)

async def run_agent(
    user_prompt: str,
    *,
    enable_reflection: bool = True,

```

```

inject_draft: str | None = None,
trace_path: Path | None = None,
scenario_id: str | None = None,
) -> dict[str, Any]:
    server_params = StdioServerParameters(
        command=sys.executable,
        args=[str(ROOT / "server.py")],
        cwd=str(ROOT),
    )
    trace_lines: list[str] = []
    explicit_emp_id = extract_explicit_employee_id(user_prompt)

    def emit(block: str) -> None:
        print(block, flush=True)
        trace_lines.append(block)

    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()
            tools = (await session.list_tools()).tools

            def complete(prompt: str) -> str:
                return llm.chat(
                    [
                        {"role": "system", "content": build_system_prompt(tools)},
                        {"role": "user", "content": prompt},
                    ],
                    temperature=0.0,
                    format="json",
                )

            async def call_tool(name: str, arguments: dict[str, Any]) -> str:
                result = await session.call_tool(name, arguments)
                return _observation_text(result)

            emit(f"=== User request ===\n{user_prompt}")
            if explicit_emp_id:
                emit(
                    f"=== Host EMP-id lock ===\n"
                    f"Using explicit {explicit_emp_id}; find_employee blocked."
                )
            emit(
                f"=== MCP tools ({len(tools)}) ===\n"
                + ", ".join(tool.name for tool in tools)
            )

            loop_lines = await run_react(
                user_prompt,
                list(tools),
                complete,
                call_tool,
                explicit_employee_id=explicit_emp_id,
                emit=emit,
            )
            policy_decision, policy_rule = policy_from_lines(loop_lines)
            tools_called = tools_called_from_lines(loop_lines)
            decision, reason = last_decision(loop_lines)
            draft_decision = decision
            draft_reason = reason
            draft_text = f"Final Decision: {decision}\nReason: {reason}"

            if inject_draft:
                emit("=== Injected bad draft (reflection demo) ===")
                emit(inject_draft)
                draft_text = inject_draft

```

```

        injected = parse_injected_draft(inject_draft)
        if injected.get("kind") == "final":
            draft_decision = injected["decision"]
            draft_reason = injected.get("reason") or draft_reason

    reflection_note = ""
    if enable_reflection and decision:
        emit("=== Reflection ===")
        reflected = run_reflection(
            user_request=user_prompt,
            draft=draft_text,
            observations=_observations_for_reflection(loop_lines),
        )
        emit(reflected["raw"])
        reflection_note = reflected.get("critique") or reflected.get("reflection") or ""
        checkpoint = reflection_checkpoint(
            draft=draft_decision,
            policy=policy_decision,
            action=reflected.get("action"),
        )
        label = str(checkpoint["label"])
        if label == "proceed":
            emit(
                "=== Reflection outcome ===\n"
                f"PROCEED draft={draft_decision!r}"
            )
            reason = reflection_note or reason
        elif label == "deny":
            emit("=== Reflection outcome ===\nDENY")
            reason = reflection_note or reason
        elif label == "escalate":
            emit(
                "=== Reflection outcome ===\n"
                "ESCALATE for human review"
            )
            reason = reflection_note or "Reflection could not confirm the draft."
        elif label == "lock":
            emit(
                "=== Decision lock ===\n"
                f"Snapping reflection {reflected.get('action')!r} "
                f"→ {checkpoint['decision']!r} "
                f"(evaluate_request rule={policy_rule})."
            )
            if policy_rule:
                reason = f"evaluate_request rule={policy_rule}"
        else:
            emit(
                "=== Reflection outcome ===\n"
                "unparsed reply; draft kept"
            )
        if label != "unparsed" and checkpoint.get("decision"):
            decision = str(checkpoint["decision"])
        if checkpoint["flag"] and "flag_for_human_review" not in tools_called:
            evaluated = _action_arguments(loop_lines, "evaluate_request")
            flag_args = {
                "employee_id": str(
                    evaluated.get("employee_id") or explicit_emp_id or ""
                ),
                "request": str(evaluated.get("item") or "equipment"),
                "reason": "Reflection escalated; human review required.",
            }
            emit("Action: " + _call_key("flag_for_human_review", flag_args))
            observation = await call_tool("flag_for_human_review", flag_args)
            emit("Observation:\n" + observation)
            tools_called = [*tools_called, "flag_for_human_review"]

```

```

        reason = _ensure_policy_rule(reason, policy_rule)
        emit(f"Final Decision: {decision}")
        emit(f"Reason: {reason}")
    else:
        snapped = bool(policy_decision and decision != policy_decision)
        if snapped:
            emit(
                "=== Decision lock ===\n"
                f"Snapping {decision!r} → {policy_decision!r} "
                f"(evaluate_request rule={policy_rule})."
            )
            decision = policy_decision
            if policy_rule:
                reason = f"evaluate_request rule={policy_rule}"
            updated = _ensure_policy_rule(reason, policy_rule)
            if snapped or updated != reason:
                reason = updated
                emit(f"Final Decision: {decision}")
                emit(f"Reason: {reason}")
            else:
                reason = updated

        saved = _save_trace(
            user_prompt,
            trace_lines,
            decision,
            path=trace_path,
            extra_header={
                **({"scenario_id": scenario_id} if scenario_id else {}),
                "draft_decision": str(draft_decision),
                "policy_decision": str(policy_decision or ""),
                "policy_rule": str(policy_rule or ""),
                "reflection": "on" if enable_reflection else "off",
            },
        )
        emit(f"=== Trace saved ===\n{saved}")

    return {
        "decision": decision,
        "reason": reason,
        "draft_decision": draft_decision,
        "draft_reason": draft_reason,
        "policy_decision": policy_decision,
        "policy_rule": policy_rule,
        "reflection": reflection_note,
        "tools_called": tools_called,
        "trace_path": str(saved),
        "steps": sum(1 for line in loop_lines if line.startswith("Thought:")),
    }

```

```

def main() -> None:
    parser = argparse.ArgumentParser(description="IT equipment ReAct agent (Ollama + MCP)")
    parser.add_argument(
        "prompt",
        nargs="?",
        default=(
            "EMP-101: My laptop is about 4 years old and slowing down during builds; "
            "can I get a replacement laptop?"
        ),
        help="Natural-language equipment request",
    )
    parser.add_argument(
        "--reflect",
        action=argparse.BooleanOptionalAction,

```

```

        default=True,
        help="Run reflection after the draft (default: on)",
    )
    parser.add_argument(
        "--inject-draft",
        default=None,
        help="Replace draft with this text before reflection (demo correction)",
    )
    parser.add_argument(
        "--trace-path",
        default=None,
        help="Optional path to write the trace file",
    )
    args = parser.parse_args()
    result = asyncio.run(
        run_agent(
            args.prompt,
            enable_reflection=args.reflect,
            inject_draft=args.inject_draft,
            trace_path=Path(args.trace_path) if args.trace_path else None,
        )
    )
    if not result.get("decision"):
        raise SystemExit(1)

if __name__ == "__main__":
    main()

```

`llm.py`

```

[python]
"""Thin Ollama /api/chat wrapper for the ReAct agent."""

from __future__ import annotations

import os
from typing import Any

import httpx
from dotenv import load_dotenv

load_dotenv()

DEFAULT_HOST = "http://localhost:11434"
DEFAULT_MODEL = "qwen3:8b"

def ollama_host() -> str:
    return os.getenv("OLLAMA_HOST", DEFAULT_HOST).rstrip("/")

def ollama_model() -> str:
    return os.getenv("OLLAMA_MODEL", DEFAULT_MODEL)

def chat(
    messages: list[dict[str, str]],
    *,
    model: str | None = None,
    temperature: float = 0.0,
    timeout_s: float | None = None,
    format: str | None = None,

```

```

) -> str:
    """Send a chat completion to Ollama and return the assistant message content.

    ``temperature`` defaults to 0 so a request stays on one tool path.
    ``think`` is false so Qwen does not spend the reply on a thinking trace.
    Pass ``format="json"`` when the reply must be a single JSON object.
    """
    host = ollama_host()
    chosen_model = model or ollama_model()
    if timeout_s is None:
        timeout_s = float(os.getenv("OLLAMA_TIMEOUT_S", "120"))

    payload: dict[str, Any] = {
        "model": chosen_model,
        "messages": messages,
        "stream": False,
        "think": False,
        "options": {"temperature": temperature},
    }
    if format:
        payload["format"] = format

    url = f"{host}/api/chat"
    try:
        with httpx.Client(timeout=timeout_s) as client:
            response = client.post(url, json=payload)
            response.raise_for_status()
    except httpx.ConnectError as exc:
        raise RuntimeError(
            f"Cannot reach Ollama at {host}. "
            "Set OLLAMA_HOST (e.g. http://localhost:11434 or "
            "http://host.docker.internal:11434) and ensure the model is pulled."
        ) from exc
    except httpx.HTTPStatusError as exc:
        raise RuntimeError(
            f"Ollama chat failed ({exc.response.status_code}): {exc.response.text}"
        ) from exc

    data = response.json()
    message = data.get("message") or {}
    content = message.get("content")
    if not content:
        raise RuntimeError(f"Ollama returned empty content: {data!r}")
    return str(content).strip()

```

`prompts.py`

```

[python]
"""System / reflection prompts for the IT equipment ReAct agent."""

from __future__ import annotations

from typing import Any

SYSTEM_PREAMBLE = """You are an IT equipment request agent. Each turn you think, then act.

thought is your own reasoning for this turn. Say what the request asks,
what the trace already shows,
and which fact is still missing. Then choose the next action from that reasoning.
Do not use a stock phrase.

Decide the next action yourself from the request and the trace.
Call a tool when a fact you need is still missing.

```

Finish only when the trace supports the decision.

Do not repeat a tool call that already has an observation.

Do not invent policy, dates, or intervals.

evaluate_request is the evidence for approve, deny, or escalate.

On finish, copy that tool's decision and its rule.

Use the EMP-### in the request when one is present.

If the user gives only a name, call find_employee first.

Never pass a person's name as employee_id.

Call flag_for_human_review only when evaluate_request returns escalate,

and call it before you finish.

Do not finish until evaluate_request is in the trace.

When the decision is escalate, do not finish until flag_for_human_review has returned.

The reason must include evaluate_request.rule and the supporting facts from that tool (item, last issued date, years elapsed, role interval) when the tool returned them.

Each turn, reply with one JSON object and nothing else. No markdown fences.

Keys are "thought", "action", "action_input", "decision", and "reason".

action is "finish", or a tool name from the tools list.

action_input is the tool arguments object, or {} when action is finish.

decision is "approve", "deny", or "escalate" only when action is finish. Otherwise null.

reason is a sentence only when action is finish. Otherwise null.

finish is not a tool name. Never put approve, deny, or escalate in action.

Tool call:

```
{"thought": "The request names EMP-101 and a laptop. The trace has no evaluate_request yet.",
"action": "evaluate_request",
"action_input": {"employee_id": "EMP-101", "item": "laptop", "reason": "slow"},
"decision": null, "reason": null}
```

Finish:

```
{"thought": "evaluate_request returned approve under within_policy.",
"action": "finish", "action_input": {}, "decision": "approve",
"reason": "within_policy plus the facts from evaluate_request"}
```

```
def format_tools_for_prompt(tools: list[Any]) -> str:
```

```
    """Render MCP tool schemas into a compact block for the system prompt."""
```

```
    lines = ["Available MCP tools:"]
```

```
    for tool in tools:
```

```
        if isinstance(tool, dict):
```

```
            name = tool.get("name")
```

```
            desc = tool.get("description")
```

```
            schema = tool.get("inputSchema")
```

```
        else:
```

```
            name = getattr(tool, "name", None)
```

```
            desc = getattr(tool, "description", None)
```

```
            schema = getattr(tool, "inputSchema", None)
```

```
    desc = desc or ""
```

```
    schema = schema or {}
```

```
    first = str(desc).strip().splitlines()[0] if desc else ""
```

```
    lines.append(f"- {name}: {first}")
```

```
    if schema:
```

```
        props = schema.get("properties") or {}
```

```
        required = schema.get("required") or []
```

```
        arg_bits = []
```

```
        for key, meta in props.items():
```

```
            kind = (meta or {}).get("type", "any")
```

```
            req = "required" if key in required else "optional"
```

```
            arg_bits.append(f"{key}:{kind}({req})")
```

```
        if arg_bits:
```

```
            lines.append(f"  args: {' '.join(arg_bits)}")
```

```
    return "\n".join(lines)
```

```
def build_system_prompt(tools: list[Any]) -> str:
    return SYSTEM_PREAMBLE + "\n" + format_tools_for_prompt(tools)

REFLECTION_PROMPT = """You are an IT compliance auditor. The draft is not final.
Ask:
1. Does this draft satisfy the employee's request?
2. Does the tool evidence strictly support it?
3. Is there ambiguity, a policy mismatch, or missing data?

action is PROCEED only when the draft equals evaluate_request.decision
and the evidence supports it.
action is DENY only when evaluate_request.decision is deny.
action is ESCALATE when evaluate_request.decision is escalate,
or when the observations do not support a decision.
Do not approve a draft the tool denied.
Do not deny a draft the tool escalated.
A borderline interval without an exceptional reason is deny, not escalate.
Do not invent dates or policy numbers.
If the decision is escalate, flag_for_human_review must be in the evidence
or the host will call it.
critique must keep evaluate_request.rule and the matching facts
(item, last issued date, years elapsed, role interval).

User request:
{user_request}

Draft decision:
{draft}

Tool observations:
{observations}

Reply with one JSON object and nothing else.
Keys are "is_valid", "critique", and "action".
is_valid is true only for PROCEED.
action is "PROCEED", "DENY", or "ESCALATE".
"""
```

6. Verbatim traces (4 graded scenarios)

Clear Approval (`traces/scenarios/01\_approve.txt`)

```
[text]
prompt: EMP-101: My laptop is about 4 years old and slowing down during builds; can I get a replacement laptop?
model: qwen3:8b
host: http://host.docker.internal:11434
decision: approve
scenario: 01_approve
draft_decision: approve
policy_decision: approve
policy_rule: within_policy
reflection: on

=== User request ===
EMP-101: My laptop is about 4 years old and slowing down during builds; can I get a replacement laptop?
=== Host EMP-id lock ===
Using explicit EMP-101; find_employee blocked.
=== MCP tools (6) ===
get_employee_info, find_employee, get_policy_limits, check_request_eligibility, evaluate_request, flag_for_human_review
```

Request: EMP-101: My laptop is about 4 years old and slowing down during builds; can I get a replacement laptop?

Thought: The request names EMP-101 and a laptop. The trace has no evaluate_request yet.

Action: evaluate_request {"employee_id": "EMP-101", "item": "laptop", "reason": "slow"}

Observation:

```
{
  "ok": true,
  "decision": "approve",
  "rule": "within_policy",
  "eligibility": {
    "ok": true,
    "eligible": true,
    "decision_hint": "approve",
    "exceeds_frequency": false,
    "employee_id": "EMP-101",
    "role": "Engineering",
    "item": "laptop",
    "last_issued": "2022-09-01",
    "next_eligible_date": "2025-09-01",
    "owned_count": 1,
    "policy": {
      "max_qty": 1,
      "min_years": 3
    },
    "years_since_last": 4.08,
    "reasons": [
      "Last laptop issued 2022-09-01; 4.08 years elapsed meets 3-year Engineering minimum."
    ]
  },
  "reasons": [
    "Last laptop issued 2022-09-01; 4.08 years elapsed meets 3-year Engineering minimum."
  ],
  "has_exceptional_justification": false
}
```

[ROUTE] evaluate_request returned approve (rule=within_policy).

Thought: evaluate_request returned approve under within_policy. The trace supports the decision.

Action: finish {}

Final Decision: approve

Reason: within_policy plus the facts from evaluate_request

=== Reflection ===

```
{
  "is_valid": true,
  "critique": {
    "rule": "within_policy",
    "item": "laptop",
    "last_issued": "2022-09-01",
    "years_elapsed": 4.08,
    "role_interval": "Engineering",
    "policy_min_years": 3
  },
  "action": "PROCEED"
}
```

=== Reflection outcome ===

PROCEED draft='approve'

Final Decision: approve

Reason: {'rule': 'within_policy', 'item': 'laptop', 'last_issued': '2022-09-01', 'years_elapsed': 4.08, 'role_interval': 'Engineering', 'policy_min_years': 3}

Clear Denial (`traces/scenarios/02\_deny.txt`)

[text]

prompt: EMP-102: I am in Sales. My laptop is only about 6 months old and I want a second/replacement laptop because mine feels slow in meetings. No damage.

model: qwen3:8b

```

host: http://host.docker.internal:11434
decision: deny
scenario: 02_deny
draft_decision: deny
policy_decision: deny
policy_rule: exceeds_frequency
reflection: on

=== User request ===
EMP-102: I am in Sales. My laptop is only about 6 months old and I want a second/replacement laptop because mine feels slow i
n meetings. No damage.
=== Host EMP-id lock ===
Using explicit EMP-102; find_employee blocked.
=== MCP tools (6) ===
get_employee_info, find_employee, get_policy_limits, check_request_eligibility, evaluate_request, flag_for_human_review
Request: EMP-102: I am in Sales. My laptop is only about 6 months old and I want a second/replacement laptop because mine fee
ls slow in meetings. No damage.
Thought: The request names EMP-102 and a laptop. The trace has no evaluate_request yet.
Action: evaluate_request {"employee_id": "EMP-102", "item": "laptop", "reason": "slow"}
Observation:
{
  "ok": true,
  "decision": "deny",
  "rule": "exceeds_frequency",
  "eligibility": {
    "ok": true,
    "eligible": false,
    "decision_hint": "deny",
    "exceeds_frequency": true,
    "employee_id": "EMP-102",
    "role": "Sales",
    "item": "laptop",
    "last_issued": "2026-04-01",
    "next_eligible_date": "2030-04-01",
    "owned_count": 1,
    "policy": {
      "max_qty": 1,
      "min_years": 4
    },
    "years_since_last": 0.5,
    "reasons": [
      "Request exceeds frequency: last laptop issued 2026-04-01; only 0.50 years elapsed, policy requires 4 for Sales. Call e
valuate_request with the free-text reason to apply exceptional-justification rules."
    ]
  },
  "reasons": [
    "Request exceeds frequency: last laptop issued 2026-04-01; only 0.50 years elapsed, policy requires 4 for Sales. Call e
valuate_request with the free-text reason to apply exceptional-justification rules.",
    "No exceptional justification in reason; clear deny under frequency policy."
  ],
  "has_exceptional_justification": false
}
[ROUTE] evaluate_request returned deny (rule=exceeds_frequency).
Thought: evaluate_request returned deny under exceeds_frequency. The request is denied due to the policy requiring 4 years be
tween laptop issues, and only 0.5 years have elapsed.
Action: finish {}
Final Decision: deny
Reason: exceeds_frequency plus the facts from evaluate_request
=== Reflection ===
{
  "is_valid": false,
  "critique": {
    "rule": "exceeds_frequency",
    "item": "laptop",
    "last_issued": "2026-04-01",

```

```

    "years_elapsed": 0.5,
    "role_interval": "Sales",
    "policy_min_years": 4
  },
  "action": "DENY"
}
=== Reflection outcome ===
DENY
Final Decision: deny
Reason: {'rule': 'exceeds_frequency', 'item': 'laptop', 'last_issued': '2026-04-01', 'years_elapsed': 0.5, 'role_interval': 'Sales', 'policy_min_years': 4}

```

Escalation A — early + damage (`traces/scenarios/03\_escalate\_damage.txt`)

[text]

```

prompt: EMP-103: I am in Engineering. My laptop was issued about a year ago but the screen was crushed on a client site / stolen bag incident – broken screen. I need an early replacement.
model: qwen3:8b
host: http://host.docker.internal:11434
decision: escalate
scenario: 03_escalate_damage
draft_decision: escalate
policy_decision: escalate
policy_rule: early_refresh_exceptional
reflection: on

=== User request ===
EMP-103: I am in Engineering. My laptop was issued about a year ago but the screen was crushed on a client site / stolen bag incident – broken screen. I need an early replacement.
=== Host EMP-id lock ===
Using explicit EMP-103; find_employee blocked.
=== MCP tools (6) ===
get_employee_info, find_employee, get_policy_limits, check_request_eligibility, evaluate_request, flag_for_human_review
Request: EMP-103: I am in Engineering. My laptop was issued about a year ago but the screen was crushed on a client site / stolen bag incident – broken screen. I need an early replacement.
Thought: The request names EMP-103 and a laptop. The trace has no evaluate_request yet.
Action: evaluate_request {"employee_id": "EMP-103", "item": "laptop", "reason": "broken screen"}
Observation:
{
  "ok": true,
  "decision": "escalate",
  "rule": "early_refresh_exceptional",
  "eligibility": {
    "ok": true,
    "eligible": false,
    "decision_hint": "deny",
    "exceeds_frequency": true,
    "employee_id": "EMP-103",
    "role": "Engineering",
    "item": "laptop",
    "last_issued": "2025-10-01",
    "next_eligible_date": "2028-10-01",
    "owned_count": 1,
    "policy": {
      "max_qty": 1,
      "min_years": 3
    },
    "years_since_last": 1.0,
    "reasons": [
      "Request exceeds frequency: last laptop issued 2025-10-01; only 1.00 years elapsed, policy requires 3 for Engineering. Call evaluate_request with the free-text reason to apply exceptional-justification rules."
    ]
  },
}

```

```

"reasons": [
  "Request exceeds frequency: last laptop issued 2025-10-01; only 1.00 years elapsed, policy requires 3 for Engineering. Call evaluate_request with the free-text reason to apply exceptional-justification rules.",
  "Free-text reason contains exceptional justification; escalate via flag_for_human_review rather than auto-deny."
],
"has_exceptional_justification": true,
"next_action": "flag_for_human_review"
}
[ROUTE] evaluate_request returned escalate (rule=early_refresh_exceptional).
Thought: evaluate_request returned escalate under early_refresh_exceptional. The next step is to flag the request for human review.
Action: flag_for_human_review {"employee_id": "EMP-103", "reason": "broken screen", "request": "laptop"}
Observation:
{
  "ok": true,
  "flagged": true,
  "queue_size": 1,
  "entry": {
    "timestamp": "2026-10-03T00:09:38.616467+00:00",
    "employee_id": "EMP-103",
    "request": "laptop",
    "reason": "broken screen"
  },
  "message": "Request flagged for human review."
}
Thought: evaluate_request returned escalate under early_refresh_exceptional. The next step is to flag the request for human review. flag_for_human_review has been called and confirmed.
Action: finish {}
Final Decision: escalate
Reason: early_refresh_exceptional plus the facts from evaluate_request
=== Reflection ===
{
  "is_valid": false,
  "critique": {
    "rule": "early_refresh_exceptional",
    "item": "laptop",
    "last_issued": "2025-10-01",
    "years_elapsed": 1.0,
    "role_interval": "Engineering",
    "policy_min_years": 3
  },
  "action": "ESCALATE"
}
=== Reflection outcome ===
ESCALATE for human review
Final Decision: escalate
Reason: {'rule': 'early_refresh_exceptional', 'item': 'laptop', 'last_issued': '2025-10-01', 'years_elapsed': 1.0, 'role_interval': 'Engineering', 'policy_min_years': 3}

```

Escalation B — non-standard item (`traces/scenarios/04\_escalate\_nonstandard.txt`)

[text]

```

prompt: EMP-104: I am in Management. Please order an ergonomic split keyboard (or standing desk) – not a standard catalog laptop/monitor/chair.
model: qwen3:8b
host: http://host.docker.internal:11434
decision: escalate
scenario: 04_escalate_nonstandard
draft_decision: escalate
policy_decision: escalate
policy_rule: non_standard_item
reflection: on

```

```

=== User request ===
EMP-104: I am in Management. Please order an ergonomic split keyboard (or standing desk) – not a standard catalog laptop/monitor/chair.
=== Host EMP-id lock ===
Using explicit EMP-104; find_employee blocked.
=== MCP tools (6) ===
get_employee_info, find_employee, get_policy_limits, check_request_eligibility, evaluate_request, flag_for_human_review
Request: EMP-104: I am in Management. Please order an ergonomic split keyboard (or standing desk) – not a standard catalog laptop/monitor/chair.
Thought: The request names EMP-104 and asks for an ergonomic split keyboard. The trace has no evaluate_request yet.
Action: evaluate_request {"employee_id": "EMP-104", "item": "ergonomic split keyboard", "reason": "health"}
Observation:
{
  "ok": true,
  "decision": "escalate",
  "rule": "non_standard_item",
  "eligibility": {
    "ok": true,
    "eligible": false,
    "decision_hint": "escalate",
    "exceeds_frequency": false,
    "employee_id": "EMP-104",
    "role": "Management",
    "item": "ergonomic_split_keyboard",
    "last_issued": null,
    "next_eligible_date": null,
    "owned_count": 0,
    "reasons": [
      "Item 'ergonomic_split_keyboard' is not on the standard catalog (laptop, monitor, ergonomic_chair); requires human review."
    ]
  },
  "reasons": [
    "Item 'ergonomic_split_keyboard' is not on the standard catalog (laptop, monitor, ergonomic_chair); requires human review."
  ],
  "has_exceptional_justification": false,
  "next_action": "flag_for_human_review"
}
[ROUTE] evaluate_request returned escalate (rule=non_standard_item).
Thought: evaluate_request returned escalate due to the non_standard_item rule. The next action is to flag the request for human review.
Action: flag_for_human_review {"employee_id": "EMP-104", "reason": "health", "request": "ergonomic split keyboard"}
Observation:
{
  "ok": true,
  "flagged": true,
  "queue_size": 1,
  "entry": {
    "timestamp": "2026-10-03T00:09:56.473924+00:00",
    "employee_id": "EMP-104",
    "request": "ergonomic split keyboard",
    "reason": "health"
  },
  "message": "Request flagged for human review."
}
Thought: The request has been escalated and flagged for human review. The trace now supports the decision to finish.
Action: finish {}
Final Decision: escalate
Reason: non_standard_item rule from evaluate_request plus the flag_for_human_review confirmation.
=== Reflection ===
{
  "is_valid": false,
  "critique": {
    "rule": "non_standard_item",

```

```

    "item": "ergonomic_split_keyboard",
    "last_issued": null,
    "years_elapsed": 0,
    "role_interval": "Management"
  },
  "action": "ESCALATE"
}
=== Reflection outcome ===
ESCALATE for human review
Final Decision: escalate
Reason: {'rule': 'non_standard_item', 'item': 'ergonomic_split_keyboard', 'last_issued': None, 'years_elapsed': 0, 'role_interval': 'Management'}
```

7. Reflection step log

Reflection correction demo (injected bad draft → corrected decision):

[text]

```

prompt: EMP-101: My laptop is about 4 years old and slowing down; can I get a replacement laptop?
model: qwen3:8b
host: http://host.docker.internal:11434
decision: approve
scenario: 00_reflection_correction
draft_decision: deny
policy_decision: approve
policy_rule: within_policy
reflection: on

=== User request ===
EMP-101: My laptop is about 4 years old and slowing down; can I get a replacement laptop?
=== Host EMP-id lock ===
Using explicit EMP-101; find_employee blocked.
=== MCP tools (6) ===
get_employee_info, find_employee, get_policy_limits, check_request_eligibility, evaluate_request, flag_for_human_review
Request: EMP-101: My laptop is about 4 years old and slowing down; can I get a replacement laptop?
Thought: The request names EMP-101 and a laptop. The trace has no evaluate_request yet.
Action: evaluate_request {"employee_id": "EMP-101", "item": "laptop", "reason": "slow"}
Observation:
{
  "ok": true,
  "decision": "approve",
  "rule": "within_policy",
  "eligibility": {
    "ok": true,
    "eligible": true,
    "decision_hint": "approve",
    "exceeds_frequency": false,
    "employee_id": "EMP-101",
    "role": "Engineering",
    "item": "laptop",
    "last_issued": "2022-09-01",
    "next_eligible_date": "2025-09-01",
    "owned_count": 1,
    "policy": {
      "max_qty": 1,
      "min_years": 3
    },
    "years_since_last": 4.08,
    "reasons": [
      "Last laptop issued 2022-09-01; 4.08 years elapsed meets 3-year Engineering minimum."
    ]
  },
  "reasons": [
```

```

    "Last laptop issued 2022-09-01; 4.08 years elapsed meets 3-year Engineering minimum."
  ],
  "has_exceptional_justification": false
}
[ROUTE] evaluate_request returned approve (rule=within_policy).
Thought: evaluate_request returned approve under within_policy. The trace now supports the decision.
Action: finish {}
Final Decision: approve
Reason: within_policy plus the facts from evaluate_request
=== Injected bad draft (reflection demo) ===
Final Decision: deny
Reason: I am guessing the laptop is too new without trusting the tools.
=== Reflection ===
{
  "is_valid": false,
  "critique": {
    "rule": "within_policy",
    "item": "laptop",
    "last_issued": "2022-09-01",
    "years_elapsed": 4.08,
    "role_interval": "Engineering",
    "policy_min_years": 3
  },
  "action": "ESCALATE"
}
=== Decision lock ===
Snapping reflection 'ESCALATE' → 'approve' (evaluate_request rule=within_policy).
Final Decision: approve
Reason: evaluate_request rule=within_policy

```

Additional reflection validation also appears inside each scenario trace under the === Reflection === sections.

8. Green CI pipeline (screenshot)

✓ CI unit tests passed (pytest 27)

```
GitHub Actions CI – Unit tests (business logic)
Repo: vidhikansara26/w7_mcp
Branch: feat/grade-polish
Workflow: .github/workflows/ci.yml
Status: PASSED (local equivalent; replace after Actions green)

--- workflow ---
name: CI

on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    name: Unit tests (business logic)
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Set up Python 3.11
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"
          cache: pip

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt

      - name: Run business logic unit tests
        run: pytest test_business_logic.py -v

      # Intentionally no Ollama / agent / MCP stdio smoke in CI.

--- pytest test_business_logic.py -v ---
test_business_logic.py::test_find_employee_name_only PASSED [ 18%]
test_business_logic.py::test_find_employee_not_found PASSED [ 22%]
test_business_logic.py::test_get_policy_limits_engineering PASSED [ 25%]
test_business_logic.py::test_get_policy_limits_unknown_role PASSED [ 29%]
test_business_logic.py::test_eligible_approve_engineering_laptop_four_years PASSED [ 33%]
test_business_logic.py::test_deny_sales_laptop_six_months PASSED [ 37%]
test_business_logic.py::test_exceeds_frequency_early_engineering_for_agent_escalation PASSED [ 40%]
test_business_logic.py::test_non_standard_item_escalate_hint PASSED [ 44%]
test_business_logic.py::test_missing_tenure_data_escalate PASSED [ 48%]
test_business_logic.py::test_eligibility_invalid_employee PASSED [ 51%]
test_business_logic.py::test_management_chair_never_owned_approve PASSED [ 55%]
test_business_logic.py::test_management_second_monitor_under_max_qty PASSED [ 59%]
test_business_logic.py::test_evaluate_approve_engineering_laptop PASSED [ 62%]
test_business_logic.py::test_evaluate_deny_sales_laptop_no_exception PASSED [ 66%]
test_business_logic.py::test_evaluate_escalate_early_refresh_exceptional PASSED [ 70%]
test_business_logic.py::test_evaluate_escalate_non_standard_item PASSED [ 74%]
test_business_logic.py::test_evaluate_escalate_missing_tenure PASSED [ 77%]
test_business_logic.py::test_evaluate_early_without_exception_stays_deny PASSED [ 81%]
test_business_logic.py::test_evaluate_unknown_employee PASSED [ 85%]
test_business_logic.py::test_evaluate_requires_nonempty_reason PASSED [ 88%]
test_business_logic.py::test_has_exceptional_justification_phrases PASSED [ 92%]
test_business_logic.py::test_flag_for_human_review_queue_and_timestamp PASSED [ 96%]
test_business_logic.py::test_flag_for_human_review_requires_fields PASSED [100%]

===== 27 passed in 0.01s =====
```